OS Unit-4 Orange Problem

Date: 21/4/26
Name: Rahul B
SRN: PES2UG24CS393

GitHub Link: https://github.com/RahulBiju-dev/PES2UG24CS393-pes-vcs

---

Phase 5: Branching and Checkout

Q5.1: Implementation of pes checkout <branch>
To implement `pes checkout <branch>`, the following changes must occur:
1. .pes/HEAD Update: The file `.pes/HEAD` must be updated to contain the reference to the target branch (e.g., `ref: refs/heads/<branch>`).
2. Working Directory Synchronization: The files in the working directory must be updated to match the state of the tree pointed to by the target branch's latest commit. This involves:
   * Deleting files that exist in the current branch but not in the target branch.
   * Updating existing files to match the versions in the target branch.
   * Adding files that exist in the target branch but not in the current one.
3. Index Update: The staging area (index) must be reset to match the tree of the new branch's commit.

Complexity: The operation is complex because it must handle "dirty" files (uncommitted changes). A naive implementation might overwrite work that hasn't been saved. It also needs to be efficient, ideally only updating files that differ between the branches rather than performing a full wipe and reload.

Q5.2: Detecting "Dirty Working Directory" Conflicts
A conflict occurs if a file has uncommitted changes that would be overwritten by the checkout. This can be detected using the index and object store:
1. Check for Local Changes: Compare each file in the working directory with its entry in the `index`. If the `mtime` or `size` (or SHA hash) differs, the file is "dirty."
2. Compare with Target Tree: For any "dirty" file, check if its hash in the current HEAD's tree is different from its hash in the target branch's tree.
3. Conflict Rule: If a file is dirty (WD != Index) AND it needs to be changed for the checkout (Current Tree != Target Tree), then `checkout` must refuse to proceed to prevent data loss.

Q5.3: Detached HEAD State
A "Detached HEAD" occurs when HEAD points directly to a commit hash instead of a branch reference.
* Committing in this state: Commits will succeed, and HEAD will update to the new commit's hash. However, since no branch (like `main`) points to these commits, they have no human-readable name.

* Recovery: If the user switches away to another branch, these commits become "dangling." They can be recovered by looking at the `reflog` (which tracks where HEAD has been) or by manually identifying the hashes in the object store and creating a new branch pointing to them (`pes branch <name> <hash>`).

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Reachability Algorithm (Mark and Sweep)
To find unreachable objects, a Mark and Sweep algorithm is used:
1. Mark Phase:
   * Initialize a "reachable" set (using a Hash Set or Bitset for efficiency).
   * Start from all references: all files in `.pes/refs/heads/`, the `.pes/HEAD` file, and any tags.
   * For each reference, recursively traverse the object graph:
     * Commit objects: Add hash to "reachable" set, then visit the parent commit and the root tree.
     * Tree objects: Add hash to "reachable" set, then visit all blobs and subtrees listed inside.
     * Blob objects: Add hash to "reachable" set.
2. Sweep Phase: Iterate through every object in `.pes/objects/`. If its hash is not in the "reachable" set, delete it.

Estimation: For 100,000 commits and 50 branches, you would visit at least 100,000 commit objects. Depending on the number of files and how much they change, you might visit several hundred thousand trees and blobs. However, because of deduplication, many commits share the same trees and blobs, significantly reducing the unique object count.

Q6.2: Concurrent GC Hazards
Running GC concurrently with a commit is dangerous due to race conditions:
* Scenario: A `pes commit` process creates a new blob object (calculates hash, writes to disk). Before it can write the commit object that *references* this blob, the GC process runs.
* The Race: The GC sees the new blob on disk but finds no reference to it in any existing branch or commit. It identifies the blob as "unreachable" and deletes it. The commit process then finishes writing the commit object, which now points to a non-existent blob (corruption).
* Git's Solution: Git uses a grace period (cooldown). `git prune` defaults to only deleting objects that haven't been modified/accessed in the last 2 weeks (using `mtime`). This ensures that "in-flight" objects created by concurrent processes are not prematurely deleted.

Screenshots

Phase 1:

1a:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2:

2a:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2b:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ xxd .pes/objects/84/07d44796e641272f4f9088b724d00d0d7cd59cbcb865b698efb8ca7317199f | head -20
00000000: 626c 6f62 2039 3132 3700 2f2f 2069 6e64  blob 9127.// ind
00000010: 6578 2e63 20e2 8094 2053 7461 6769 6e67  ex.c ... Staging
00000020: 2061 7265 6120 696d 706c 656d 656e 7461   area implementa
00000030: 7469 6f6e 0a2f 2f0a 2f2f 2054 6578 7420  tion.//.// Text
00000040: 666f 726d 6174 206f 6620 2e70 6573 2f69  format of .pes/i
00000050: 6e64 6578 2028 6f6e 6520 656e 7472 7920  ndex (one entry
00000060: 7065 7220 6c69 6e65 2c20 736f 7274 6564  per line, sorted
00000070: 2062 7920 7061 7468 293a 0a2f 2f0a 2f2f   by path):.//.//
00000080: 2020 203c 6d6f 6465 2d6f 6374 616c 3e20     <mode-octal>
00000090: 3c36 342d 6368 6172 2d68 6578 2d68 6173  <64-char-hex-has
000000a0: 683e 203c 6d74 696d 652d 7365 636f 6e64  h> <mtime-second
000000b0: 733e 203c 7369 7a65 3e20 3c70 6174 683e  s> <size> <path>
000000c0: 0a2f 2f0a 2f2f 2045 7861 6d70 6c65 3a0a  .//.// Example:.
000000d0: 2f2f 2020 2031 3030 3634 3420 6131 6232  //   100644 a1b2
000000e0: 6333 6434 6535 6636 2e2e 2e20 2031 3639  c3d4e5f6...  169
000000f0: 3939 3030 3030 3020 3432 2052 4541 444d  9900000 42 READM
00000100: 452e 6d64 0a2f 2f20 2020 3130 3036 3434  E.md.//   100644
00000110: 2066 3765 3864 3963 3062 3161 322e 2e2e   f7e8d9c0b1a2...
00000120: 2020 3136 3939 3930 3031 3030 2031 3238    1699900100 128
00000130: 2073 7263 2f6d 6169 6e2e 630a 2f2f 0a2f   src/main.c.//./
```

Phase 3:

3a:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
  staged:     index.c

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  test_objects.c
  untracked:  instructions.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  README.md
  untracked:  pes.h
  untracked:  tree.h
  untracked:  index.h
  untracked:  pes.c
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  test_sequence.sh

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776756785 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776756785 6 file2.txt
100644 8407d44796e641272f4f9088b724d00d0d7cd59cbcb865b698efb8ca7317199f 1776756313 9127 index.c
```

3b:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776756785 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776756785 6 file2.txt
100644 8407d44796e641272f4f9088b724d00d0d7cd59cbcb865b698efb8ca7317199f 1776756313 9127 index.c
```

Phase 4:

4a:

```
Author: PES User <pes@localhost>
Date:   1776756869

    Add farewell

commit d582dd162dc4fac31aae26c4d438132d29c6ae42c27e68d48564e29eb8185631
Author: PES User <pes@localhost>
Date:   1776756869

    Add worldell

commit 94ddacf2fb43b579a6336ec427b6841544f23122c17ab4d2d53885a0019a3e14
Author: PES User <pes@localhost>
Date:   1776756869

    Initial committial commiQ
```

4b:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/0f5005bdaae62e101b831fa7cc345b54f618b5a5f5c8e5d4fb997c02dc5ede
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/3c/30f7d279e5b00226a4bacec5b80e3d240d1ee3fc1a953af4dc878af96a0163
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/84/07d44796e641272f4f9088b724d00d0d7cd59cbcb865b698efb8ca7317199f
.pes/objects/89/f984a70991cf83e99b38cad0bff5313540e23a4303413c5a098048b348e7c4
.pes/objects/94/ddacf2fb43b579a6336ec427b6841544f23122c17ab4d2d53885a0019a3e14
.pes/objects/b3/28545471d0d7e9b150425ac667e5a488e46e53ded79e8dd1f025bfb5487233
.pes/objects/d5/82dd162dc4fac31aae26c4d438132d29c6ae42c27e68d48564e29eb8185631
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4c:

```
rahulb@rahul-VM-Ubuntu2204:~/PES2UG24CS393-pes-vcs$ cat .pes/refs/heads/main
b328545471d0d7e9b150425ac667e5a488e46e53ded79e8dd1f025bfb5487233
```